COMP24011 Lab1: Reversi

Ian Pratt-Hartmann and Francisco Lobo

Academic session: 2026-27

Introduction

In this assignment, you will write a Java program to play the game of reversi, sometimes known by its trademark name of Othello. Reversi is played on an 8 by 8 board, initially set up as shown in Figure 1 on the left, with the players moving alternately by placing their own pieces (black or white) on the board, one at a time. By default Black always goes first.

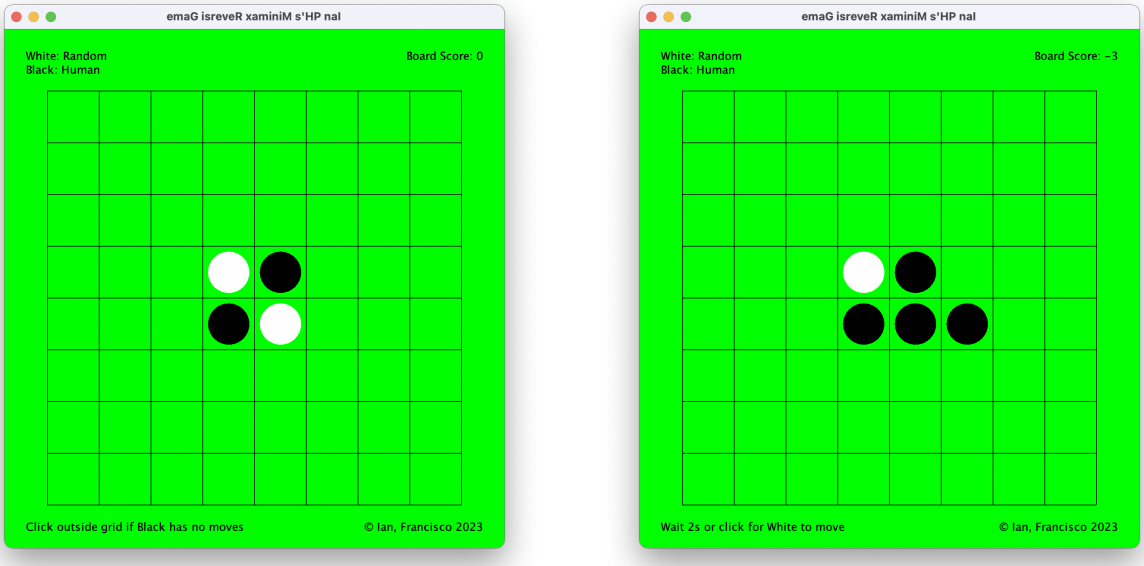


Figure 1: The opening position in a reversi game; Black moves to $(5, 4)$.

In the `lab1` branch of your `COMP24011_2026` GitLab repo you will find the Java code for the game of reversi in Figure 1. To compile and run this program you can use the following commands

```
$ javac *.java
$ java Othello
```

In addition to the game board, the main window also shows

- on the top left, the player agents that control the white and black pieces
- on the top right, the current board score
  i.e. the difference between the number of white and black pieces
- on the bottom left, a message indicating who's turn it is and how to proceed with the game
- on the bottom right, a copyright notice; please do not distribute the code for this assignment.

Note that the board score value will be positive when White is winning and negative when Black is winning. By default the Java program allows a human player (always Black) to play against a computer agent that moves randomly (always White). You can change the code in `Othello.java` to configure these game settings. In fact, you will **need to** do this to test your player agent!

The graphical interface of the Java reversi game is not very responsive and a bit basic, to put it mildly. For example:

- The human agent waits for the user to click on a square and then makes that move. There is a 2 seconds delay before non-human agents can move, but the user can also click to request the agent to move.
- Clicking on squares which do not represent legal moves just produces an irritating beep.
- If it is the turn of the human player and there is no legal move, then the user must click anywhere in the green area outside the grid of the board, to generate a skip move and allow play to pass to the other player.
- The interaction between mouse click events from the user and timer delay events that trigger non-human agents is not perfect... It's better not to click while the computer is 'thinking' ☺
- If you want to play another game, you need to close the window and re-run the program.

But these are details (which you are welcome to improve, though no extra marks are available). By default, as the game progresses there is some debug output printed on the console. If you close the window after making the move shown in Figure 1 on the right, the terminal will display

```
$ java Othello
Loading game...
Human: Move (5,4), valid, 0.2 secs, no error
Current boardState: {........|........|........|...wb...|...bbb..|........|....
....|........}
Current boardScore: -3
```

This reports that the user attempted to move to the square $(5, 4)$, that this move was valid and hence made. The program times how long the agent took to select a move in response to the mouse or timer event, in increments of 0.2 seconds, and if this generated any runtime error. Note that x and y move coordinates can only have values from 0 to 7, and the top left square is at $(0, 0)$.

When you close the window, an ASCII representation of the current state of the game is written. It shows the squares of the board row-by-row, with the letters 'w' and 'b' representing white and black pieces respectively, and dots representing empty squares. This representation is enclosed by curly braces or square brackets depending on whose turn it is to move, White or Black respectively. The Java program accepts these ASCII representations as an optional parameter on the command-line. If you execute

```
$ java Othello "{........|........|........|...wb...|...bbb..|........|........
|........}"
```

you will restart the game exactly where you've left off. You'll probably find this very useful while testing and debugging the Java code for your agent.

Let's review how reversi is played. In the following description, a *line segment* is a sequence of board squares forming a contiguous straight line (horizontal, vertical or diagonal). The rule for a player to place a piece is:

- The piece must be placed on an empty square such that there is a line segment starting at the piece played, passing through one or more pieces of the opposing colour, and ending on a piece of the player's own colour.

When such a line segment exists, we say that the opponent's pieces on that line segment are *bracketed*. When a piece is played, all bracketed pieces change colour (necessarily, to player's own colour). Note that, in many situations, there may be more than one line segment of bracketed pieces; in that case, all of those the bracketed pieces change colour.

In Figure 2, the left-hand picture shows a game state where $(3, 5)$ is one of the possible moves for White. This move brackets three of the opponent's pieces, namely $(4, 5)$ horizontally, $(3, 4)$ vertically, and $(4, 4)$ diagonally, resulting in the position shown in the right-hand picture.

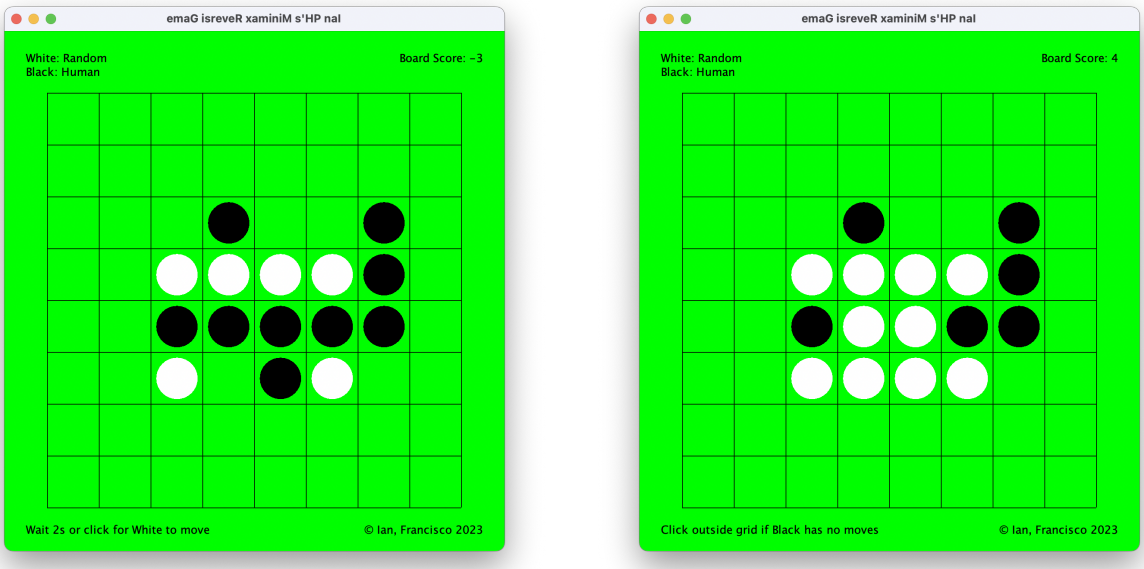


Figure 2: A move by White in a reversi game, and its result.

If, and only if, a player cannot move, but his opponent can, he misses a turn. The game ends when neither player can move. This usually, but not always, happens because all the squares have been occupied. The winner is the player with the greater number of pieces of his own colour on the board; if there is no such player, the result is a draw.

Assignment

Try out the Java reversi game, and play a few games. You will see that the computer is playing terribly. In fact, it computes the possible moves in the current position, and then simply selects one at random! Your first task is to implement a player agent that uses the minimax search with alpha-beta pruning to play a better game.

Before starting this assignment, make sure you understand generic types in Java, e.g. things like `ArrayList<SomeClass>`. Otherwise, the Java code is very simple. To complete this assignment you do not need to understand the code in `OthelloDisplay.java` or `MoveStats.java` which runs the game UI and the collection of debug stats.

The main game logic is in the class `BoardState`. An instance of this class represents a situation in the game. The field `colour` encodes whose turn it is (1 for White; -1 for Black), while the low-level methods `getContents(int x,int y)` and `setContents(int x,int y,int piece)` allow retrieval and setting on the individual board squares. Here, a value of 1, -1 or 0 represents presence of, respectively, a white piece, a black piece or no piece at all at the square on rank x and file y.

A separate class `Move` is used to encode the actual moves; it has two public fields, `x` and `y`; as well as a public method `isSkip()`. As long as `isSkip()` is false, `x` and `y` give the coordinates of a newly placed piece in the obvious way. If, on the other hand, `isSkip()` is true, then the move represented is the act of 'skipping one's go' (passing control to the opponent); and the values of the coordinate fields have no meaning. Note that an instance of `Move` is just a move — not necessarily a legal one. In particular, the skip move is legal if and only if the current player cannot put a piece down, but his opponent can.

Both of these classes include the code necessary to be used as Java `HashMap` keys. To make things easier for you, the `BoardState` class has the method `checkLegalMove(Move m)`, which checks

whether it is possible for the current player to make the given move, as well as the method `makeLegalMove(Move m)`, which actually executes the move. In fact, to make things *really* easy, we have provided the method `getLegalMoves()` that returns the list of all and only those legal moves for the current player. Notice that this list cannot be empty as long as the game is not over.

All game player agents correspond to implementations of the abstract Java class `MoveChooser`. The rest of the control is handled by the program. You are given 4 working implementations of this class:

- `MoveChooserHuman` that simply relays the moves chosen by the user on the UI.
- `MoveChooserFirst` is a computer player that always selects the first available legal move.
- `MoveChooserRandom` is a computer player that selects one of available legal moves at random.
- `MoveChooserMonteCarlo` is a computer player that selects one of available legal moves using Monte Carlo tree search.

You can control which one of these implementations is used for each player by commenting out the appropriate lines in `Othello.java`. For example, you could set

```
whiteChooser= new
    //MoveChooserAlphaBeta(searchLimit)
    MoveChooserMonteCarlo(searchLimit/2)
    //MoveChooserRandom(randomSeed)
    //MoveChooserFirst()
    //MoveChooserHuman()
    ;
```

recompile everything, and run `java Othello` to test if you can win against the MonteCarlo agent.

Part 1: AlphaBeta agent

To complete this part of this assignment, you need to implement the `MoveChooserAlphaBeta` computer player. The sample code that you're provided compiles but doesn't work. You cannot change the public interface of the sample code; the parameters for each method are documented in the source.

The constructor `MoveChooserAlphaBeta(int searchLimit)` needs to set a name for your player agent (to be used on the UI), and record the search depth that will control the minimax search algorithm. The method `chooseMove(BoardState boardState, Move hint)` is the most important that you need to implement in this class. This is called when it's the computer's turn to move in the board state `boardState`. As you're coding a non-human player, the 2nd parameter giving the move `hint` of the triggering UI event should be ignored. All you have to do is write a move selection procedure that uses the minimax algorithm with $\alpha\beta$-pruning discussed in the lectures. Note that you are allowed to code auxiliary functions to achieve this. The depth of the minimax search should be controlled by the `searchLimit` field of your agent's instance.

We want you to proceed in 2 stages. In the first stage you implement a board evaluation function, using the following method. Each square in the playing area is assigned a number as follows.

```
120 -20 20  5  5 20 -20 120
-20 -40 -5 -5 -5 -5 -40 -20
 20  -5 15  3  3 15  -5  20
  5  -5  3  3  3  3  -5   5
  5  -5  3  3  3  3  -5   5
 20  -5 15  3  3 15  -5  20
-20 -40 -5 -5 -5 -5 -40 -20
120 -20 20  5  5 20 -20 120
```

These numbers reflect the value for a player of being on the respective square. Note that almost all squares on the edges have high value (since pieces here are hard to take) and squares in the corners have the highest value (since pieces here cannot be taken). By contrast, neighbouring squares have negative values, since a piece there will allow one's opponent to move onto a high-value square. (Incidentally, the above array of values was taken from an older textbook by Peter Norvig.) The value of a board position can then be defined by adding up the weights of all those squares occupied by white pieces and subtracting the weights of those squares occupied by black pieces. Thus, the value for the board is always counted from White's point of view. This value needs to be computed by the method `boardEval(BoardState boardState)` of your `MoveChooserAlphaBeta` implementation.

In the second stage you implement `MoveChooserAlphaBeta.chooseMove()` so that it performs minimax search with $\alpha\beta$-pruning. If you find $\alpha\beta$-pruning hard, you may prefer first to try ordinary minimax, and then graduate to $\alpha\beta$-pruning once that's working. Your first challenge is to calculate minimax values for board positions. Note that your `MoveChooserAlphaBeta.boardEval()` can be used when you reach the limit of the search depth. This is controlled by the field `searchLimit` of the `Othello` class, which by default is set to 6. You may wish to set it to a lower value to speed up development, and increase it when you get everything working.

It is worth thinking about what you'll need to do when searching through a tree of board positions. Remember that `BoardState` instances are Java objects. By calling `BoardState.makeLegalMove()` on these, you will update the board (and the current player). Therefore, if you want to get the daughters of a vertex `b0` in the search tree, you will need to create a fresh copy of this board state for each legal move, and then execute the move in question on that copy. We have provided a cloning method `BoardState.deepCopy()`, so that a fresh copy of the board state `b0` can be made with the call `b1 = b0.deepCopy()`; subsequently modifying `b1` (e.g. by executing a legal move) does not then change `b0`. Note that if your agent runs into the 10 seconds timeout set in `MoveStats` for the collection of debug stats, then you probably have written inefficient code.

When you start coding, change `Othello.java` so that `MoveChooserFirst` plays Black and your `MoveChooserAlphaBeta` plays White with `searchLimit` set to 4. Use the ASCII representations to help you debug any illegal moves or runtime errors that your code may generate. Once your player agent is beating `MoveChooserFirst`, it should be playing a decent game of reversi against a human. Try this out!

After that, increase `searchDepth` to 6 and have `MoveChooserRandom` play Black. If your implementation is correct, your agent should consistently beat a player choosing moves at random, and should not take more than a few seconds for any move when run on an ordinary PC at this search depth. Now try setting the search depth to 8. This should be enough to thrash most human players, and won't be too slow.

Part 2: MonteCarlo agent

This part of the assignment involves Monte Carlo tree search (MCTS). The implementation of the `MoveChooserMonteCarlo` computer player uses the algorithm explained in the lectures, except that the selection of vertices for exploration in the search tree does not work properly. (You will need to fix this, as explained below.) For the moment, try the algorithm out by setting one of the players to `MoveChooserMonteCarlo` and the other to (your implementation of) `MoveChooserAlphaBeta`. You will probably find that even this version of MCTS does pretty well. Note that the constructor `MoveChooserMonteCarlo(int mctsSeconds)` takes as parameter the number of seconds the agent spends exploring the game tree before making a decision.

Look at the code in `MoveChooserMonteCarlo`, and try to understand how it (nearly) implements the algorithm given in the lectures. Notice that `MoveChooserMonteCarlo.chooseMove()` creates an object `root` of the abstract Java class `BoardTreeVertex`. Every object of this type has a field `children`, which is an array of objects of that same class `BoardTreeVertex`. The resulting tree with root `root` represents the tree of explored vertices mentioned in the lectures.

The abstract class `BoardTreeVertex` already contains the method required for backpropagation, as well as the method `getBestChild()`, which selects the most promising daughter of the current vertex to be followed in the exploration phase. The way `getBestChild()` works is simple: it calls the method `vertexEval(BoardTreeVertex childVertex)` of the `BoardTreeVertex` object to calculate a value for each daughter, and then selects at random among the *lowest* values. In many descriptions of MCTS, you will see that exploration selects the daughter with highest value. In our implementation, however, the value of a vertex is the value *for that vertex*, which is controlled by the opponent player and thus is opposite to the value for the mother; hence our selection of a lowest-value daughter. (But this is just an implementation detail.)

Note that `MoveChooserMonteCarlo` cannot create objects of the abstract `BoardTreeVertex` class. Instead its tree objects are made using the concrete `BoardTreeSolution` class, which implements the abstract `BoardTreeVertex.vertexEval()` method. The sample code that you're provided simply returns the dummy constant value of 666. This means that, in the implementation you have been given, exploration is totally random! This fact notwithstanding, MCTS plays a fairly decent game against minimax. The standard way to calculate the value of the daughter v with mother u is

$$\left(w_v/n_v - \gamma \sqrt{2 \log(n_u/n_v)} \right), \tag{1}$$

where n_v is the number of games played by the daughter, w_v the number of wins of the daughter, and n_u the number of games played by the mother. The so-called *exploration* constant γ is generally taken to be $\sqrt{2}$.

Your task is simply to correct the dummy implementation of the `BoardTreeVertex.vertexEval()` method with the correct the calculation according to the expression (1). This should be quite a routine piece of re-coding once you have got your head around what is going on. By the way, the exploration constant γ is defined by the field `explorationConstant` of the `BoardTreeVertex` class. (You should use this constant in your code, not a magic number.)

Submission & Mark Scheme

The solution to this assignment consists of two source files called `MoveChooserAlphaBeta.java` and `BoardTreeSolution.java` which must be submitted to your CS GitLab repo with the tag `lab1_sol`. The deadline for submission is **14:00 on Friday 23rd October**. Please follow the `README.md` instructions in your `COMP24011_2026` GitLab repo to obtain the `lab1` materials and complete your submission.

The lab exercise will be auto-marked **offline**. The automarker program will download your submission from CS GitLab and test it against our AlphaBeta and MonteCarlo solution agents, respectively called `MoveChooserReferenceAB` and `MoveChooserReferenceMCTS`. In each test, your agent plays the reversi game against another agent starting on a random board position for a total of 20 *game moves* (or until the game is over). The following statistics are collected.

- The change in board score. If the board score changes in favour of the player your agent controls, then your agent **beats** its opponent in that test.
- The number of times your agent **completes** the required `chooseMove()` call within the 10 seconds time limit of `MoveStats`. If your agent times out and fails to complete a move, then `MoveChooserFirst.chooseMove()` will be used to decide your player's turn on that occasion.

- The number of times your AlphaBeta agent performs minimax search with $\alpha\beta$-pruning **correctly**. This happens only when `MoveChooserReferenceAB.chooseMove()` *could choose* the same move for the particular board state *and* chosen search depth.
- The number of times your AlphaBeta agent calculates the board evaluation value **correctly**. This happens only when it returns the same value as `MoveChooserReferenceAB.boardEval()` for the particular board state.
- The number of times the MonteCarlo agent calculates tree vertex values **correctly** using your implementation of the `BoardTreeSolution` class. This happens only when it returns the same value as the `vertexEval()` implementation used in `MoveChooserReferenceMCTS` for the particular tree vertex.
- The time your agent takes to complete *correct moves*, in increments of 0.2 seconds, will be compared to the time the reference agents take to complete the same moves. The number of times your agent is at least **as fast as** the reference agent is counted.

A total of 20 marks is available in this assignment. The marking scheme is as follows.

2 marks Your agent will be initialised as `MoveChooserAlphaBeta(4)` and play 4 tests as White against `MoveChooserRandom()`. You get 0.5 marks for each game your agent beats its opponent (or else makes only correct moves).

2 marks Your agent will be initialised as `MoveChooserAlphaBeta(8)` and play 4 tests as White against `MoveChooserReferenceAB(5)`. You get 0.5 marks for each game your agent beats its opponent (or else makes only correct moves).

4 marks Your agent will be initialised as `MoveChooserAlphaBeta(8)` and play 2 tests as White and 2 tests as Black against `MoveChooserReferenceAB(7)`. You get 1 mark for each game your agent beats its opponent (or else makes only correct moves).

4 marks The MonteCarlo agent will be initialised as `MoveChooserMonteCarlo(5)` and use your `BoardTreeSolution` implementation to play 2 tests as White and 2 tests as Black against `MoveChooserReferenceMCTS(2)`. You get 1 mark for each game your agent beats its opponent (or else makes only correct moves).

2 marks If your agent is as fast as the reference agent while playing the above 16 test games. To obtain 1 mark your implementation needs to be fast 50% of the time.

2 marks If your agent correctly calculates board/vertex evaluation values while playing the above 16 test games. To obtain 1 mark your implementation needs to be correct 50% of the time.

4 marks If your agent correctly performs minimax search with $\alpha\beta$-pruning while playing the above 12 AlphaBeta test games. To obtain 3 marks your implementation needs to be correct 80% of the time. To obtain 2 marks your implementation needs to be correct 65% of the time. To obtain 1 mark your implementation needs to be correct 50% of the time.

Additional Clarifications

- This lab exercise is fully auto-marked. If you submit code which does not compile on the Unix image of the Kilburn lab machines, you will score 0 marks. We recommend that you make a backup of the Java source files provided for this lab, and before submission you check that your submission files compile with the original lab sources.
- If your code produces runtime errors under ordinary game conditions, then it will fail to complete moves. The automarker will try to finish test games by deferring the broken call to the `MoveChooserFirst` implementation, as described above. Such turns will contribute negatively when calculating the percentage of time your implementation was correct. In case your agent becomes unresponsive, the automarker enforces a time limit of 150 seconds per test game; this is sufficient to allow your agent 10 seconds for each move.